

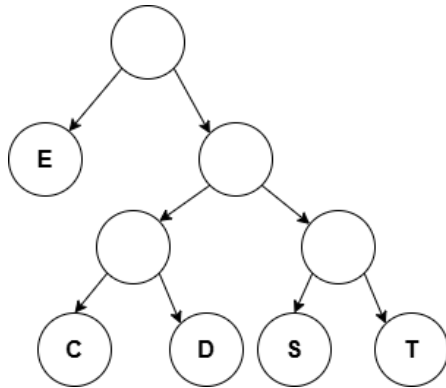
CSE 280 Challenge Set 07

(c) BYU-Idaho

Question 1

Part 1

Consider the encoding tree below. How would you encode SEED ?



Part 2

How many bits did you save with the encoding assuming letters take 8 bits?

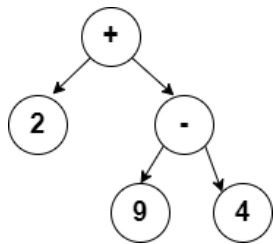
Question 2

Part 1

Create a binary tree that represents the following mathematical expression written in in-fix notation:

$$(2 + (9 - 4)) * (8 / (4 - 2))$$

Note that you only need to put numbers and operators in the tree. Parentheses are not needed because they are implied by the parent/child relationship in the tree. For example, the tree for the first two operations on the left hand side could be drawn as:



Verify that the forward in-order traversal of the tree matches the expression above (with the implied parentheses).

Part 2

Determine the pre-order traversal of the tree you created in Part 1. This is the pre-fix notation (or Polish Notation) of the expression. Put spaces in between each number and operator.

Part 3

Determine the post-order traversal of the tree you created in Part 1. This is the post-fix notation (or Reverse Polish Notation) of the expression. Put spaces in between each number and operator.

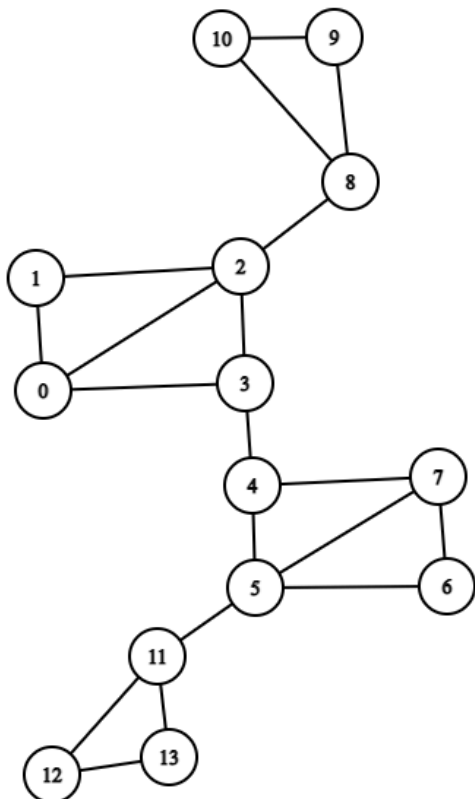
Part 4

Goto the website: <https://www.rpn-calc.com/> and enter in your Reverse Polish Notation (spaces in between each number and operator) and verify the answer is correct (solve the expression in Part 1). Why are parentheses not required?

Question 3

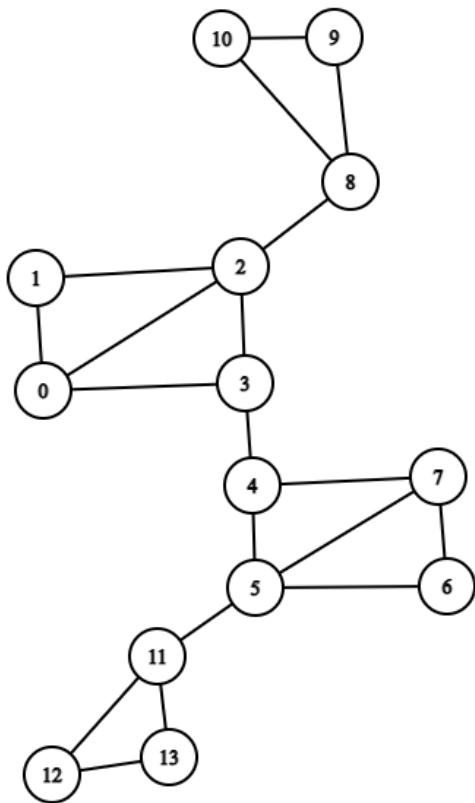
Part 1

In the graph below, highlight or mark a spanning tree by using Depth First Search. Start at node 0.



Part 2

In the same graph below, highlight or mark a spanning tree by using Breadth First Search. Start at node 0.



Part 3

Draw both spanning trees rooted at vertex 0 next to the graphs above. Compare the heights of the two trees.

Part 4

Suppose that each vertex in the graph has the ability to transmit (or forward) a message to all adjacent vertices. If I wanted to transmit a message from vertex 0 to all the vertices in the original graph, should I create a spanning tree using BFS or DFS? Why?

Question 4

Visit the following online maze solver: <https://zyrridian.github.io/bfs-dfs-maze/>

Answer the following questions:

- Generate new mazes (big or small) and solve them using BFS or DFS (or at the same time). Set the speed slow enough so you can see the behavior of BFS and DFS. Note that the maze can be represented as a graph of allowable movements. Which algorithm most commonly found the end of the maze the fastest?
- Which algorithm used the most computer memory?
- Consider writing a solver for Sudoku. You would have 9 possible numbers (open paths in the maze) to put in each of the boxes not yet populated. If a number choice leads to breaking a rule, then we can't move forward on that path (wall in the maze). How might memory impact our decision of whether to use BFS or DFS?